

Scraping HTML with Beautiful Soup (CodeGrade)

 [_ \(https://github.com/learn-co-curriculum/python-p4-web scraping-beautiful-soup\)](https://github.com/learn-co-curriculum/python-p4-web scraping-beautiful-soup)  [_ \(https://github.com/learn-co-curriculum/python-p4-web scraping-beautiful-soup/issues/new\)](https://github.com/learn-co-curriculum/python-p4-web scraping-beautiful-soup/issues/new)

Learning Goals

- Introduce web scraping and its uses.
 - Retrieve data from an HTML document through web scraping.
-

Key Vocab

- **Web scraping:** Web scraping, web harvesting, or web data extraction is data scraping used for extracting data from websites.
 - **HTML:** The HyperText Markup Language or HTML is the standard markup language for documents designed to be displayed in a web browser.
 - **CSS:** Cascading Style Sheets is a style sheet language used for describing the presentation of a document written in a markup language such as HTML or XML.
 - **CSS selector:** CSS selectors define the elements to which a set of CSS rules apply.
-

Introduction

Web scraping is the act of parsing a web page's HTML and pulling, or "scraping" pertinent data from that HTML. In this reading, we'll take a brief look at what scraping is and how to accomplish it.

A more thorough code-along is coming up next, but if you would like to follow along, `lib/scrapper.py` is provided for you.

You can use the `pipenv install && pipenv shell` command to enter the virtual environment to run your code.

What is Scraping and Why Use it?

As we established above, scraping is a technique used to grab data out of the HTML that makes up a web page. Scraping can be difficult to accomplish—in order to get the data we want, we need to closely examine the HTML and identify exactly which elements contain the information we're interested in. It requires a high degree of precision.

So, if scraping is so tricky, why do we use it? Well, not all of the data we might be interested in using to program is available to use through APIs. For example, let's say we're creating an app that catalogs popular musicians and searches the web for their upcoming concerts. A quick Google search will reveal that, unfortunately for us, there isn't a "Popular Musician" API out there just waiting to be used. There is, however, a very comprehensive list of musicians on the Billboard website. In such a scenario, you may want to programmatically grab every musician's name from the Billboard list and store those artists in your own database.

Here's another example: let's say you're creating an app that allows a user to subscribe to a news feed. You anticipate that your users are super-tech savvy and might be interested in subscribing to some lesser-known tech news sites. Such sites may not have an API that makes their articles available to you. Instead, you would have to scrape those sites for their latest news articles and send those newest articles to your users.

These are just a few examples of situations in which scraping might come in handy. Now that we have a few use-cases that illustrate the utility of scraping, let's talk about *how* to scrape.

Scraping HTML Using BeautifulSoup and requests

Refresher: What is requests?

[requests](https://requests.readthedocs.io/en/latest/)  (<https://requests.readthedocs.io/en/latest/>) is a module in Python that allows us to programmatically make HTTP requests. It gives us a bunch of useful methods to make different types of requests, but for this guide, we're only interested in one right now: `get`. This method takes one argument, a URL, and will return to us the HTML content of that URL.

In other words, running:

```
html = requests.get("https://google.com/")
```

stores the HTML of Google into a variable, `html` . (More specifically, it actually stores the HTML in a temporary file that we can then call read on to get the raw HTML. We won't worry about that here though.)

What is BeautifulSoup?

Beautiful Soup is a Python package for parsing HTML and XML documents. It creates a parse tree for parsed pages that can be used to extract data from HTML, which is useful for web scraping. Put simply, BeautifulSoup takes in HTML and spits out a collection of objects we can get information from.

The HTML that would normally be rendered as a webpage can be scraped with BeautifulSoup into a many small pieces. BeautifulSoup makes the level of precision required to extract the necessary data much easier to attain.

Beautiful Soup transforms a complex HTML document into a complex tree of Python objects. But you'll only ever have to deal with about four kinds of objects: `Tag` , `NavigableString` , `BeautifulSoup` , and `Comment` .

Let's get BeautifulSoup up and running and look at a very basic example of its usage. Then, we'll move on to the next lesson, where you'll try it out for yourself.

Installing BeautifulSoup

Installing BeautifulSoup is as easy as `pipenv install bs4` (we've already done that for you here).

Opening a Web Page as HTML with BeautifulSoup and requests

Let's say we have a file, `scraper.py` which is responsible for (you guessed it) scraping. We need to require BeautifulSoup and requests:

```
from bs4 import BeautifulSoup
import requests
```

more code coming soon!

We can use the following line to grab the HTML that makes up the Flatiron School's landing page at flatironschool.com :

```
html = requests.get("https://flatironschool.com/", headers=headers)
```

If you get a 403 Forbidden error, the website may be trying to prevent bots. You may need to define headers with `user-agent` .

```
headers = {'user-agent': 'my-app/0.0.1'}
html = requests.get("https://flatironschool.com/", headers=headers)
```

Next, we'll create a BeautifulSoup object to take the string of HTML returned by requests's `get` method which represents the document as a nested data structure.

Let's save the HTML document in a variable, `doc` that we can then operate on:

```
doc = BeautifulSoup(html.text, 'html.parser')
```

If we were to `print` out `doc` right now, we'd see something like this in our terminal:

```
<!DOCTYPE html>

<html lang="en-US">
<head>
<!-- Google Tag Manager -->
<script>(function(w,d,s,l,i){w[l]=w[l]||[];w[l].push({'gtm.start':
    new Date().getTime(),event:'gtm.js'});var f=d.getElementsByTagName(s)[0],
    j=d.createElement(s),dl=l!='dataLayer'?'&l='+l:'';j.async=true;j.src=
    'https://www.googletagmanager.com/gtm.js?id='+i+dl;f.parentNode.insertBefore(j,f);
    })(window,document,'script','dataLayer','GTM-KZZ9JB');</script>
<!-- End Google Tag Manager -->
```

```
<meta charset="utf-8"/>
<meta content="width=device-width, initial-scale=1" name="viewport"/>
<link href="https://gmpg.org/xfn/11" rel="profile"/>
<link color="#00b3e6" href="https://flatironschool.com/wp-content/themes/flatiron-school/assets/images/favicons/safec
<link href="https://flatironschool.com/wp-content/themes/flatiron-school/assets/images/favicons/favicon.ico" rel="sh
<meta content="https://flatironschool.com/wp-content/themes/flatiron-school/assets/images/favicons/browserconfig.xml
...

```

If you look through further, you can find the `body` with lots of content.

```
<body class="home page-template-default page page-id-5 wp-embed-responsive">
<!-- Google Tag Manager (noscript) -->
<noscript><iframe height="0" src="https://www.googletagmanager.com/ns.html?id=GTM-KZZ9JB" style="display:none;visibi
<!-- End Google Tag Manager (noscript) -->
<a class="visually-hidden-focusable" href="#siteMain">Skip to content</a>
<!--Site Header-->
<header class="site-header is-absolute site-header-white" id="siteHeader" role="banner">
<div class="header-container">
<h1 class="site-logo logo-light">
<span class="visually-hidden">Flatiron School</span>
</h1>
<nav class="navigation-collapse" id="primaryNavigation">
<ul class="navbar navbar-primary">
...

```

And on, and on. It is a *lot* to go through, and it can also look pretty messy and difficult to read. But don't worry! BeautifulSoup will help us parse this. What we're looking at here is all of the HTML that makes up the web page found at [www.flatironschool.com](http://flatironschool.com/)  (<http://flatironschool.com/>). The massive lines above are actually a snapshot of that HTML converted into a structure of nested nodes by BeautifulSoup.

What are Nested Nodes?

Nested nodes refers to any tree of elements in which parent elements branch off to contain children elements. In fact, we've seen similarly nested structures before when we dealt with nested data structures like Dictionaries. By creating a nested structure, BeautifulSoup allows us to do things like iterate over a collection of elements from the HTML document and use brackets, `[]`, and dot notation to access elements within the nested structure.

Using BeautifulSoup to Extract Data

Note: For this reading, we'll be using the Flatiron School website. However, how you scrape a page is **very specific to the content of the page you are trying to scrape**. That means that if the webpage you wrote certain scraping code for ever changes, your scraping code will likely no longer work correctly. So, the Flatiron School website that this reading refers to **may have changed!** Some of the examples here may be specific to an earlier version of the site and won't work look or work exactly as shown when you try them out on your own. That's okay though. Just follow along with the reading and, if you want to try it out, feel free to use the examples provided to guide you in scraping content that is present on the page.

Visit [this Flatiron School link](http://flatironschool.com/)  (<http://flatironschool.com/>) and use your browser's developer tools to inspect the page. (You can just right-click anywhere on the page and select "inspect element".)

You should see something like this:



```
<!--hero-->
▼<section class="hero-image-block image-block image-static">
```

The [element inspector](https://developers.google.com/web/tools/chrome-devtools/inspect-styles/)  (<https://developers.google.com/web/tools/chrome-devtools/inspect-styles/>) view on the bottom half of the page is revealing all of the page's HTML to us! In fact, the HTML it is showing us is *exactly the same* as the HTML `print` out to our terminal with the help of BeautifulSoup and requests `get` .

Now that we understand what BeautifulSoup is and have seen how it opens the HTML that makes up a web page, let's look at how we use it to actually scrape information.

Using CSS Selectors to Get Data

Beautiful Soup allows you to use CSS selectors in order to retrieve specific pieces of information out of an HTML document.

What is a CSS Selector

In the following code:

```
<div id="my-div"></div>
```

The id and class attributes of the HTML elements are the CSS selectors. You would refer to the div with this selector: `#my-div` (using the `#` to indicate id), and the paragraph with this selector: `.my-paragraph` (using the `.` to indicate class).

Beautiful Soup's `.select` Method

Beautiful Soup's `.select` method can be called on the `doc` variable that we set equal to that giant string of HTML that BeautifulSoup retrieved for us. The `.select` method takes in an argument of the CSS selector you want to retrieve. Let's take a look.

Choosing a CSS Selector

How do we determine which selector to use to retrieve the desired information? Remember that the HTML document that BeautifulSoup retrieved for us to operate on is *exactly the same* HTML that makes up the web page. Let's go back to "www.flatironschool.com  (<http://www.flatironschool.com>)" and use the element inspector to find the selector of a certain piece of our HTML. In this case, we'll look the element containing the text '10 years of shaping tech talent':



In order to identify the CSS selector, click the button in the upper left corner of the console pane that looks like a mouse icon partially in a box.

Once activated, hover over the '10 years of shaping tech talent' text. This will highlight its HTML element for us. Notice that:

```
<h1> class="heading-financier ..." </h1>
```

is highlighted in the above image. If you click on the carrot at the left end of that line, it will open up to show you what that element contains (with lots of spacing around it):

```
"10 years of shaping tech talent"
```

We found it! That text lives in a header whose class includes `"heading-financier"`. Now we're ready to use the `.select` method to grab the text we want:

Calling the `.select` method

In our `scraper.py` file, we had the following code:

```
from bs4 import BeautifulSoup
import requests

headers = {'user-agent': 'my-app/0.0.1'}
html = requests.get("https://flatironschool.com/", headers=headers)

doc = BeautifulSoup(html.text, 'html.parser')
```

Let's call `.select` on `doc` and give it the argument of our CSS selector:

```
from bs4 import BeautifulSoup
import requests

headers = {'user-agent': 'my-app/0.0.1'}
html = requests.get("https://flatironschool.com/", headers=headers)

doc = BeautifulSoup(html.text, 'html.parser')
```

```
print(doc.select('.heading-financier'))
```

If we run the above code, the last line would return something like:

```
[<h1 class="heading-financier color-white mb-20 text-shadow animate animate-1s" data-animation="fadeInUp" data-animc
      10 years of shaping tech talent                                </h1>]
```

Although dense, it is possible to figure some things out. First of all, `doc.select('.heading-financier')` returned what looks like an Array containing one `ResultSet` object. In actuality, this 'ResultSet' is *also* a special `Beautiful Soup` object, but works very much like an List. If you look closely at the object contained within it, you'll see that it has the '10 years of shaping tech talent' text towards the end!

```
print(doc.select('.heading-financier')[0].contents)
```

Using `.contents` allows us to access text content inside an element scraped by BeautifulSoup. It will return a list of `bs4.element.NavigableString` objects. After running the code, we'd see something like this returned:

```
['\n                                10 years of shaping tech talent                                ']
```

Note: If we add `.strip` to the end we can clean up the extra whitespace and simply return `"10 years of shaping tech talent."`

We did it! We used BeautifulSoup to get the HTML of a web page. We used the element inspector in the browser to ID the CSS selector of the data we wanted to scrape. We used the `.select` BeautifulSoup method, along with that CSS selector, to grab the element that contains our desired data. Finally, we used the `.contents` method to retrieve the desired text.

Iterating over elements

Sometimes we want to get a collection of the same elements, so we can iterate over them. For instance, a little further down the [page](https://flatironschool.com)  (<https://flatironschool.com>) are some of the courses offered by Flatiron School. We can practice iterating by trying to scrape the titles of all courses from these elements.

This time, if we hover over one of the elements containing a course, we'll see there are three classes assigned, `heading-60-black`, `color-black` and `mb-20`. Since CSS classes are often shared, we'll use all three to try and get only the content we need:

```
from bs4 import BeautifulSoup
import requests

headers = {'user-agent': 'my-app/0.0.1'}

# We will be using the our-courses page
html = requests.get("https://flatironschool.com/our-courses/", headers=headers)

doc = BeautifulSoup(html.text, 'html.parser')

print(doc.select('.heading-60-black.color-black.mb-20'))
```

Notice that each class is listed without spaces!

Even though the BeautifulSoup returns a `bs4.element.ResultSet` (which looks like a list in Python). We can iterate over the elements like a list.

```
[<h2 class="heading-60-black color-black mb-20">
    Choose Your Course
    Software Engineering
    Data Science
    Product Design
    Cybersecurity Engineering
    </h2>, <h2 class="heading-60-black color-black mb-20">
    </h2>, <h2 class="heading-60-black color-black mb-20">
    </h2>, <h2 class="heading-60-black color-black mb-20">
    </h2>, <h2 class="heading-60-black color-black mb-20">
    </h2>]
```

Instead of just outputting the results of `doc.select`, if we assign them to a variable, we can then iterate over that variable using a for loop:

```
courses = doc.select('.heading-60-black.color-black.mb-20')

for course in courses:
    print(course.contents[0].strip())
```

We'd see something like this:

```
Choose Your Course
Software Engineering
Data Science
Product Design
Cybersecurity Engineering
```

Not *exactly* the course listing as it scraped some other content as well - a great example how tricky scraping can be - but we've still looped through/iterated through html elements!

Operating on bs4.element.ResultSet

Let's take another look at the element returned to us by our call on the `.select` method. In the previous example, we had many `BeautifulSoup` objects to iterate over. Looking at just the first one:

```
courses = doc.select('.heading-60-black.color-black.mb-20')[0]
```

We get an output similar to the following:

```
<h2 class="heading-60-black color-black mb-20">
    Software Engineering
</h2>
```

This is an `bs4.element.Tag` object.

When we use BeautifulSoup methods, we can get the tag object. Looking at the output object, we can see it has a `name` attribute, "h2". We can get this info directly by adding these to the end of our `doc.select` call:

```
name = doc.select('.heading-60-black.color-black.mb-20')[0].name
# => "h2"
```

This is the name of the tag element, not to be confused with the HTML attribute 'name' that can be assigned to elements. Those types of attributes can be accessed with `.attrs`. Using `.attrs` will return ids, names, and classes, but will also return other useful content like `alt` and `src` for images.

```
doc.select('.heading-60-black.color-black.mb-20')[0].attrs
```

Since this example doesn't have any attributes besides the CSS classes, we just get back the classes we already know:

```
{'class': ['heading-60-black', 'color-black', 'mb-20']}
```

One last but important method to note is `children`. Adding `children` will return any child nodes nested *inside this element*. On a webpage, an `h2` HTML element may be nested within a `div`. When scraped, this relationship can be represented by having an object named "div" with a child object named "h2".

Beautiful Soup collects these objects into a hierarchical data structure, much like the nested lists and dictionaries we've been building and manipulating for a while now. This structure allows us to iterate over an list of BeautifulSoup objects and use iterator to grab the values of attributes and text.

Conclusion

By using BeautifulSoup, we can get any website's HTML, represented in objects, including any text or data displayed on that site. Using methods like `.select`, we can then filter out the specific parts of the website we need and use additional methods like `.contents` and `.attrs` to extract the content we want.

As each website is designed differently, scraping tends to require customized code for each site you want to scrape. As sites update their styles and designs, scrapers we've built may no longer work.

However, being able to scrape websites gives us access to information that can be time-consuming or otherwise very difficult to collect. Taking a little time to update a scraper is typically much easier and faster than manually updating data.

Note: One final note about scraping - the content we're getting by scraping is all technically publicly available, as it all visible on public websites. Be careful, however, as some content may not be used without permission or licensing. Images, for instance, often belong to someone and can have a license attached - something you would probably want to look into before scraping any image `src` attributes from a site!

Resources

- Scraping is a big topic, and it takes *a lot* of practice to get comfortable doing it. The below resource is a great place to learn more about scraping and even get some practice with simple examples. If you felt really confused by this reading, we recommend checking it out before moving on.
 - [Beautiful Soup](https://beautiful-soup-4.readthedocs.io/en/latest/)  [\(https://beautiful-soup-4.readthedocs.io/en/latest/\)](https://beautiful-soup-4.readthedocs.io/en/latest/)
 - [Requests](https://requests.readthedocs.io/en/latest/)  [\(https://requests.readthedocs.io/en/latest/\)](https://requests.readthedocs.io/en/latest/)

This tool needs to be loaded in a new browser window

Load Scraping HTML with BeautifulSoup (CodeGrade) in a new window